

Universidad Icesi

Algoritmos y Programación I

Tarea Integradora

Angela Sofía Hurtado Rosales

Especificación y Planeación

1. Caracterización del sistema

UniBot se define, bajo el enfoque sistémico, como un sistema de información transaccional y conversacional cuyo propósito es mediar entre los estudiantes de primer ingreso y el conocimiento institucional restringido a su carrera, actúa como una capa de filtrado entre la pregunta del usuario y la respuesta generada por un modelo de lenguaje. A continuación se describen sus cinco componentes fundamentales:

Componente	Descripción
Objetivo	Automatizar la atención de primer nivel a estudiantes de primer ingreso, que logre responder dudas académicas, reglamentarias y técnicas de forma restringida y pertinente a su carrera, reduciendo la carga operativa de los asesores humanos y evitando la entrega de información de otros programas.
Entradas	• Código institucional del estudiante • Contraseña maestra y única del administrador • Mensajes del estudiante en el chat • Archivo estudiantes.csv (usuarios autorizados) • Archivo carreras.json (temas permitidos por carrera) • Nuevas altas de estudiantes ingresadas por el administrador • Nuevos temas/palabras clave ingresados por el administrador.
Procesos	• Validación de credenciales de estudiante y administrador • Lectura, traducción y organización de un bloque de información de archivos planos (CSV/JSON). • Construcción del prompt restrictivo mediante la clase PromptBuilder. • Pedirle cosas a la IA en segundo plano para que no se congele la aplicación. • Registro, actualización y cierre de sesiones. • Cálculo del costo por sesión (caracteres × 2 COP) • Auditoría y cierre forzado de sesiones activas • Persistencia del historial de conversación
Salidas	• Respuestas del asesor virtual filtradas por carrera • Mensaje de bienvenida personalizado • Archivo de log de la conversación (.txt) • Reporte de auditoría de sesiones (activas/cerradas) • Archivo estudiantes.csv actualizado • Archivo carreras.json actualizado • Archivo registro_sesiones.json actualizado.

Componente	Descripción
Elementos (subsistemas)	Subsistema de Autenticación (validación de código de estudiante y contraseña maestra de administrador) • Subsistema de Gestión de Datos (lectura/escritura de CSV y JSON) • Subsistema de Interacción (menús de consola y ciclo de chat) • Subsistema de Inteligencia Artificial (PromptBuilder y LLMService) • Subsistema de Sesiones y Auditoría (control de costos, estados y trazabilidad).

Vista de conjunto: el sistema recibe entradas discretas (credenciales, mensajes, archivos de configuración), las procesa a través de sus subsistemas de forma secuencial e interdependiente, y produce salidas verificables (respuestas, logs, registros actualizados). Esta caracterización evidencia que UniBot es un sistema abierto: intercambia información constantemente con su entorno (el estudiante, el administrador y la API externa del LLM) y depende de la calidad de sus entradas (los archivos CSV/JSON) para mantener la integridad de sus salidas.

2. Modelo de proceso y marco de trabajo

2.1 Justificación de la elección metodológica

Se selecciona el Desarrollo Incremental, apoyado en prácticas ágiles de Scrum (Product Backlog, Sprints y tablero Kanban), por las siguientes razones:

- Entrega continua de valor: el enunciado ya divide el proyecto en cuatro fases funcionales y verificables (requisitos, prototipo estructurado, POO/TypeScript, integración de IA), cada una entregando software ejecutable, no solo documentación.
- Reducción de riesgo técnico: al construir primero la base estructurada en JavaScript puro y luego migrar a TypeScript y POO, los errores de lógica se detectan temprano, antes de sumarles la complejidad de tipado estricto y consumo de APIs externas.
- Adaptabilidad: si el módulo de autenticación o lectura de archivos requiere ajustes, estos se corrigen en su propio sprint sin detener el desarrollo de los módulos posteriores.
- Trazabilidad: el uso de un Product Backlog priorizado permite mapear cada requerimiento a la clase o función que lo implementa, lo cual es exigido explícitamente en la Fase 4 (matriz de trazabilidad).
- Escala apropiada para el equipo: al tratarse de un desarrollo individual o de equipo reducido, Scrum se aplica de forma simplificada, sin roles como Scrum Master dedicado, enfocándose en los artefactos (Backlog, Sprints, tablero) más que en las ceremonias.

2.2 Organización de las fases como Sprints

Cada fase definida en el enunciado se organiza como un Sprint con alcance, resultado esperado y entregable propio:

Sprint	Nombre	Alcance	Entregable principal
Sprint 0	Fundamentos	Análisis de sistemas, modelado UML (casos de uso), Product Backlog, creación del repositorio Git base con README.md.	Documento de especificación (PDF) + diagramas + tablero Kanban.
Sprint 1	Prototipo estructurado	Menú dual (Administrador / Estudiante) en JavaScript puro, autenticación vía .env, lectura del CSV con fs, ciclo de acceso del estudiante.	Código .js funcional + estudiantes.csv + .env.example + 3 commits.
Sprint 2	Programación Orientada a Objetos y TypeScript	Migración a TypeScript, clases LectorCSV, LectorJSON, Estudiante y Carrera, gestión de accesos (escritura CSV) y gestión de reglas (modificación JSON).	Código TypeScript modular + tsconfig estricto + TSDoc.
Sprint 3	Integración de IA	Clase PromptBuilder, clase LLMService (async/await), auditoría de sesión con logs en /logs, manejo robusto de errores (try/catch).	Diagrama de clases final + código completo + logs + matriz de trazabilidad.

3. Especificación de requerimientos — Product Backlog inicial

Los requerimientos funcionales están orientados a entregar valor observable al usuario final (estudiante o administrador); los no funcionales garantizan la calidad, seguridad y mantenibilidad del sistema.

ID	Historia de usuario	Tipo	Prioridad	Sprint	Criterio de aceptación
RF-01	Como estudiante, quiero ingresar mi código institucional para poder acceder al chat del asesor virtual.	Funcional	Alta	Sprint 1	El sistema valida el código contra estudiantes.csv y solo permite el acceso si existe una coincidencia exacta.
RF-02	Como estudiante, quiero chatear con el asesor virtual y recibir únicamente respuestas relacionadas con mi carrera.	Funcional	Alta	Sprint 3	El PromptBuilder inyecta los temasPermitidos de la carrera del estudiante y el bot rechaza amablemente preguntas fuera de ese listado.
RF-03	Como administrador, quiero autenticarme con la contraseña maestra para acceder al módulo de back-office.	Funcional	Alta	Sprint 1	La contraseña se valida contra una variable de entorno (.env) y nunca aparece escrita en el código fuente.
RF-04	Como administrador, quiero registrar nuevos estudiantes para mantener actualizado el listado de usuarios autorizados.	Funcional	Alta	Sprint 2	El sistema agrega una nueva línea al archivo estudiantes.csv usando fs.appendFileSync sin corromper el archivo existente.

ID	Historia de usuario	Tipo	Prioridad	Sprint	Criterio de aceptación
RF-05	Como administrador, quiero gestionar los temas permitidos por carrera para controlar el alcance de las respuestas del bot.	Funcional	Alta	Sprint 2	El sistema permite seleccionar una carrera y agregar una nueva palabra clave, sobrescribiendo correctamente carreras.json.
RF-06	Como administrador, quiero auditar las sesiones registradas para conocer cuántas están activas y cuántas cerradas.	Funcional	Media	Sprint 3	El menú de administrador muestra un conteo correcto de sesiones Activas y Cerradas leyendo registro_sesiones.json.
RF-07	Como administrador, quiero forzar el cierre de sesiones activas para depurar el sistema.	Funcional	Media	Sprint 3	Las sesiones Activas cambian su estado a Cerrada, se les calcula el costo según los caracteres acumulados y se actualiza el archivo.
RF-08	Como estudiante, quiero que mi conversación se guarde automáticamente al escribir "salir" para fines de trazabilidad.	Funcional	Media	Sprint 3	Se genera un archivo .txt con el formato AAAA-MM-DD_HH-mm-ss_CodigoEstudiante_Carrera.txt dentro de /logs.
RF-09	Como sistema, quiero calcular el costo de cada sesión para llevar control económico del uso del bot.	Funcional	Media	Sprint 3	Al cerrar una sesión, el costo se calcula como longitud total de caracteres $\times$ 2 COP y se guarda solo en ese momento.
RNF-01	Como desarrollador, quiero que la contraseña maestra nunca esté hardcodeada para proteger el acceso administrativo.	No funcional	Alta	Sprint 1	La contraseña se consume exclusivamente desde variables de entorno mediante dotenv.
RNF-02	Como usuario, quiero que el sistema maneje errores de archivos sin cerrarse abruptamente.	No funcional	Alta	Sprint 2 / 3	Toda lectura o escritura de archivos está envuelta en try/catch y, ante un error, se informa por consola sin crashear el proceso.
RNF-03	Como desarrollador, quiero identificadores de código consistentes para facilitar el mantenimiento.	No funcional	Media	Todos	El código usa CamelCase para variables/métodos y PascalCase para clases, en inglés.
RNF-04	Como desarrollador, quiero tipado estricto en TypeScript para prevenir errores en tiempo de compilación.	No funcional	Media	Sprint 2 / 3	El archivo tsconfig.json tiene strict: true y el proyecto compila sin errores de tipos.
RNF-05	Como usuario, quiero mensajes claros en consola para entender si una	No funcional	Baja	Todos	Cada operación (login, registro, búsqueda) muestra un mensaje

ID	Historia de usuario	Tipo	Prioridad	Sprint	Criterio de aceptación
	operación fue exitosa o falló.				explícito de éxito, error o acceso denegado.

Declaración uso de IA generativa:

Para el desarrollo del presente trabajo se utilizó Claude (Anthropic) como herramienta de apoyo durante el proceso de elaboración. Su utilización estuvo orientada principalmente a refinar y guiar el proceso de trabajo, facilitar la organización de las actividades y contribuir a mantener una estructura ordenada durante las diferentes etapas del proyecto. Adicionalmente, Claude fue utilizado como herramienta de apoyo para la creación de un tablero interactivo bajo la metodología Kanban, destinado exclusivamente a la organización y seguimiento de las actividades asociadas al desarrollo del proyecto. La utilización de esta herramienta se limitó a las funciones anteriormente descritas y no sustituyó el análisis, la toma de decisiones, la elaboración de contenidos ni la responsabilidad intelectual de los integrantes del equipo. Las decisiones finales y los resultados presentados en este trabajo fueron revisados y definidos por el equipo responsable del proyecto.